

COMP101

Introduction to Programming

2025-26

Lecture-17_1

Iteration Python – ‘for in’

for loop: terminology

- ‘Deterministic loop’
 - Use when we know how many times to iterate
- Uses reserved ‘in’ with an ‘iterator’
 - a sequence in a one-dimensional array of data
 - list, string, tuple, range, dictionary, set
 - 1,3,5,7,9
 - ‘cat’, ‘dog’, ‘giraffe’
 - indicates where to start and when to stop the loop

Python for loop

- Iterates over a 'sequence' ('iterable')
- Every item in the sequence has the same operation executed on it
 - efficient coding
 - execution is for a fixed number of times

for loop: other languages

- In C

```
//Output the integers 10 to 19
for( a = 10; a < 20; a = a + 1 ){
    printf("value of a: %d\n", a);
}
```

In Python

```
#output integers 10 to 19
for i in range(10,20):
    print (i)
```

We look at 'in range' in following lecture

- In Java

```
//Output the integers 0 to 4
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

In Python

```
#output integers 0 to 4
for i in (0,1,2,3,4):
    print (i)
```

for loop: Python

#output integers 0,1,2,3,4

```
for i in (0,1,2,3,4):  
    print (i)
```

#output integers 0 and 4 only

```
for i in (0,4):  
    print (i)
```

#output integers 0,1,2,3

```
for i in range(0,4):  
    print (i)
```

#see next lecture for 'in range'

Python for-loop is easier than in other languages by using 'iterables' (sequences)

i is the 'iterating var' and numbers in parentheses specify how many times to loop

We don't increment the loop counter: The iterable replaces the increment expression in other languages.

There is a small trade-off with this but not a problem when knowing how the loop behaves

There is no need to explicitly identify when the loop starts and stops, e.g. using { } etc

for loop syntax: in and in range

for <counter> in (sequence):
 <statements>

This lecture

for <counter> in range (sequence):
 <statements>

Following lecture

- ‘for’ is a reserved word – can only be used to build a for loop
- ‘counter’ is an integer – can be any name but usually i, j, k
 - must never be changed in the loop – done automatically
- ‘in’ is a reserved word to check for an item in the sequence
- ‘range’ is a built-in function to return a sequence of values
- ‘sequence’ is a sequence of values (integer, but can be string)
- ‘:’ indicates the start of the loop body follows next
- ‘statements’ in the loop body are indented after the colon

for loop syntax: in and in range

for <counter> in (sequence):
 statements

This lecture

for <counter> in range (sequence):
 statements

Following lecture

- No test condition (as in a while loop)
 - we know how many times to iterate
- No changes to loop counter after each iteration – automatic
- No single integer in a sequence – must be more than one
 - for loop always works in combination with an iterable
 - loop iterates over an item ('member', 'index', 'element') of the sequence

for loop syntax: in

```
for <COUNTER> in (sequence):  
    <STATEMENTS>
```

```
for i in (1,2,3):  
    print (i)
```

1
2
3

- A sequence can have as many items as required
 - all comma separated in parentheses
- Don't forget the colon at the end of the line
- This sequence has 3 items - loop iterates 3 times

for loop syntax: in - error

```
for <COUNTER> in (sequence):  
    <STATEMENTS>
```

```
for i in (3):  
    print(i)
```

Traceback (most recent call last):

File "D:/COMP101_2023-24/Week-05/Lectures/in_in-range.py", line 4, in <module>

for i in (3):

TypeError: 'int' object is not iterable

- the sequence only holds one value - gives error
- A sequence must have more than one value, comma separated
- for-in loops only work with a sequence

for loop: in

```
for i in (1,3,5):  
    print (i)
```

#o/p down the screen

1

3

5

```
for i in (1,3,5):  
    print (i, end=" ")
```

#o/p across the screen with spaces

1 3 5

```
for i in (1,3,5):  
    print (i, end="")
```

#o/p across the screen with no spaces

135

for loop: in

Information only for now

#use a var

```
odds = (1,3,5,7,9)
```

```
for i in (odds):
```

```
    print (i)
```

1

3

5

7

9

Gives a bit more flexibility in managing the sequence if it needs changing

(1,3,5,7,9)

- 'tuple' - sequence comma-separated in parentheses

[1,3,5,7,9]

- 'list' - sequence comma-separated in square brackets

for loop: in

```
odds = (1,3,5,7,9)
for i in (odds):
    print ("test")
```

```
test
test
test
test
test
```

```
#o/p includes counter
odds = (1,3,5,7,9)
for i in (odds):
    print ("test", i)
test 1
test 3
test 5
test 7
test 9
```

for loop: sequence index

- Items in a sequence have an associated index
 - Square brackets denote an index number
 - Default index value = [0]
- odds = (1,3,5,7,9) with its index is:

Odds:	1	3	5	7	9
Index:	[0]	[1]	[2]	[3]	[4]

for loop: in and in enumerate

#o/p sequence with its index

```
odds = (1,3,5,7,9)
```

```
for i in (odds):
```

```
    print (odds.index(i), i)
```

0 1

1 3

2 5

3 7

4 9

All items have an index

1 is at index[0]

3 is at index[1]

5 is at index[2]

7 is at index[3]

9 is at index[4]

The comma following odds.index(i), puts a space between the index number and the item

#'enumerate' o/p sequence with its index

```
odds = (1,3,5,7,9)
```

```
for i in enumerate(odds):
```

```
    print (i)
```

(0, 1)

(1, 3)

(2, 5)

(3, 7)

(4, 9)

Does the same as code on the left but uses inbuilt-function 'enumerate' to output the item index number. Code is simpler in generating a counter

Note default output includes parentheses around the index and the item, and they are comma-separated¹⁴

Enumerate

Information only

- The enumerate function takes input of an iterable and adds a counter (a number) to each element
 - It returns an 'enumerate object' – it 'enumerates the item'
 - Code is simpler and no need to explicitly code the counter
- The counter can act as an index to each element
 - Associating an item with an index number means we can indicate the position of the item in the sequence
 - can be used to reference these elements at a later stage when required
 - counter defaults to zero for the first item
 - Gives 'index-element' or 'key-value' pairs

Enumerate: integer

```
odds = (1,3,5,7,9)
```

```
for i in odds:
```

```
    print (i)
```

1

3

5

7

9

Although these elements have an index, we have not coded it to be output with the associated element

```
odds = (1,3,5,7,9)
```

```
for i in enumerate(odds):
```

```
    print (i)
```

(0, 1)

(1, 3)

(2, 5)

(3, 7)

(4, 9)

Enumerate returns an 'index-element' or 'key-value' pair format

Enumerate: string

```
language = ('Python, Java, C')  
for i in language:  
    print (i)
```

P
y
t
h
o
n
,
J
a
v
a
,
C

Sequence has 1 element – note the apostrophes. The element has an index but no index is output here, only the element

```
language = ('Python, Java, C')  
for i in enumerate(language):  
    print (i)
```

(0, 'P')
(1, 'y')
(2, 't')
(3, 'h')
(4, 'o')
(5, 'n')
(6, ',')
(7, 'J')
(8, 'a')
(9, 'v')
(10, 'a')
(11, ',')
(12, 'C')
(13, ',')

Sequence has 1 element – note the apostrophes.

A sequence has indexes starting at [0]

'in enumerate' adds the associated index number to every item including the commas as they are part of the element – they are inside the apostrophes

Enumerating allows us to easily extract individual items from the element

Enumerate: string

```
language = ('Python', 'Java', 'C')  
for i in language:  
    print (i)
```

Python
Java
C

Sequence has 3 elements – note the apostrophes.
Each element has an index but no index is output here, only the element

```
language = ('Python', 'Java', 'C')  
for i in enumerate(language):  
    print (i)
```

(0, 'Python')
(1, 'Java')
(2, 'C')

Sequence has 3 elements – note the apostrophes.
A sequence has indexes starting at [0]

'in enumerate' adds the associated index number to every element.
Note the commas are not part of any element – they are not in apostrophes

Enumerating allows us to pre-process data for easier extraction of individual items

Enumerate

```
stud_name = ('name-1', 'name-2', 'name-3')  
for i in enumerate (stud_name, 201512345):  
    print (i)
```

```
(201512345, 'name-1')  
(201512346, 'name-2')  
(201512347, 'name-3')
```

Although the enumerate statement says (stud_name, 201512345), the enumerate function always outputs the index number first before the value – hence the output.

Enumeration starts indexing at zero by default.

Can override the default and start indexing with our own value – here it is 201512345 which is incremented by 1 each time round the loop

in enumerate

```
numbers = (1,2,3,4,5)
```

```
for index,value in enumerate(numbers):
```

```
    print ('Index [' ,index, ' ] is ',value)
```

```
Index [ 0 ] is 1
```

```
Index [ 1 ] is 2
```

```
Index [ 2 ] is 3
```

```
Index [ 3 ] is 4
```

```
Index [ 4 ] is 5
```

Enumerate can access the index number and the value.

At each iteration, enumerate increments the index by 1 automatically.

We use two loop vars comma separated – one for the index and one for the value – ‘argument unpacking’ (useful in dictionary handling)

for loop: in with string

```
word = input("Enter a word: ") bat
```

```
#initialise counter to space or zero if numeric
```

```
i = ' '
```

```
for i in word:
```

```
    print (word,end=" ")
```

```
bat bat bat
```

```
print ("Loop ended")
```

```
print ("Press Return to continue")
```

```
input () #halt execution until user presses return key
```

The word 'bat' has three characters:

These index as:

b	a	t
[0]	[1]	[2]

'for' prints the word 3 times;

A 5 character word will print 5 times etc

For loop: video

- <https://www.youtube.com/watch?v=94UHCEmprCY>